FICHE MODULE 1

Comprendre la machine

L'IA ne devine pas : elle prolonge ce qu'on lui donne. Cette fiche explique ce qu'un LLM fait vraiment, et ce qu'il ne sait pas faire.

Avant tout : sortie subie ou sortie pilotée ?

Une scène classique : on demande « Rédige une user story pour améliorer la réservation », puis on corrige quatre fois (« c'est pour le mobile », « tu as oublié les premium », « plus court »…). Vingt minutes plus tard, le résultat est flou et personne ne sait expliquer ses choix.

Chaque correction révélait une information qu'on pouvait donner **dès le départ** :

Ce qu'on a dit	Ce qu'il fallait donner
« rédige une user story »	un **objectif** explicite
« c'est pour le mobile »	la **cible** et le **canal**
« tu as oublié les premium »	le **contexte** métier
« plus court »	le **format** et les **critères**

Deux façons de travailler avec l'IA, toutes deux utiles :

- **Explorer** (*vibe coding* chez les développeurs) : on tente, on observe, on jette. Idéal pour trouver des idées ou esquisser un brouillon.
- **Livrer** (*AI-driven development*) : on cadre, on contrôle, on capitalise. Indispensable pour un livrable réutilisable et partagé.

Viber pour explorer, piloter pour livrer.

Un LLM prédit la suite

Un LLM (*Large Language Model*) est entraîné sur d'énormes volumes de texte. Il produit sa réponse **token après token** : à chaque pas, il choisit un des morceaux les plus probables, avec une part de hasard.

Conséquence : il produit du **plausible**, pas du **vrai**.

Un **token** est un morceau de mot. Les mots courants tiennent en un token, les mots rares sont découpés (« anticonstitutionnellement » = 5 tokens). Le français consomme plus de tokens que l'anglais pour dire la même chose. Plus de tokens, c'est plus cher, plus lent, et plus de place prise dans la mémoire de travail du modèle.

La réponse varie, et chaque mot compte

- Relancer la même demande donne des réponses **différentes** : une réponse est une **proposition**, pas une vérité.
- Plus la demande est floue, plus les réponses divergent.

- Quelques mots (« mobile », « premium », « critères d'acceptation », « en 5 points ») agissent comme des **signaux** et peuvent changer toute la structure de la réponse.

Quand la sortie bouge beaucoup, la bonne question n'est pas « quel modèle ? » mais « **qu'est-ce que je n'ai pas cadré ?** ».

Le modèle n'a pas de mémoire

À chaque message, l'assistant **renvoie toute la conversation** au modèle. Ce que vous avez dit plus tôt continue donc de peser sur la suite, et une nouvelle conversation repart de zéro.

Modèle, assistant, harness, tools

- Le **modèle** (GPT, Claude, Gemini…) est le moteur.
- L'**assistant** (ChatGPT, Copilot, Claude.ai…) est le produit que vous utilisez. Tout le logiciel autour du modèle s'appelle le **harness** : consignes, mémoire, interface.
- Deux assistants sur le même modèle peuvent donner des résultats différents. Les noms changent tous les trimestres ; les notions restent.

Seul, un modèle a des limites : il ignore ce qui s'est passé après son entraînement (**knowledge cutoff**), calcule mal et n'a accès à aucun système de l'entreprise. L'assistant lui branche des **tools** qui compensent : recherche web, calcul, connecteurs **MCP** vers Jira, SharePoint…

La boucle agentique

Exemple : « Résume les tickets Jira ouverts sur la réservation. »

1. **Demande** : vous écrivez votre demande.
2. **Décision** : le modèle choisit d'appeler un tool.
3. **Exécution** : le harness appelle Jira, avec vos droits d'accès.
4. **Résultat** : les tickets entrent dans le contexte.
5. **Réponse** : le modèle rédige la synthèse.

Les étapes 2 à 4 se répètent tant que le modèle a besoin d'informations. Avoir des tools ne suffit pas : **modèle + tools + boucle + objectif = agent**, et c'est le harness qui fait tourner la boucle.

Trois raisons de garder la main

Limite	Exemple	Réflexe
Elle hallucine : une réponse fausse, assurée et plausible	des avocats ont cité en justice des jurisprudences inventées par l'IA	vérifier les faits et les sources
Elle amplifie vos données : données sales → réponses sales	un export de réservations sans les annulations donne une analyse trop optimiste	regarder ce qu'on lui donne
Elle ne remplace pas l'humain : confiance, négociation, empathie	elle prépare l'argumentaire d'une négociation, elle ne la mène pas	garder l'humain au contact

À retenir

1. Une réponse est une proposition : **relancez** avant de faire confiance.
2. Quand la sortie varie, cherchez ce que vous n'avez **pas cadré**.
3. Le modèle relit toute la conversation à chaque message : ce qui y traîne compte.
4. Un agent, c'est un modèle qui enchaîne lui-même des tools, en boucle.

Mots-clés : LLM, token, knowledge cutoff, assistant, harness, tools, MCP, agent, boucle agentique, hallucination — voir le glossaire.

FICHE MODULE 2

Bien demander

D'une demande floue à une sortie exploitable : quoi donner à l'IA, dans quel ordre, et comment réutiliser ce qui marche.

Les 4 ingrédients d'une bonne demande

Ingrédient	La question	Exemple
Rôle / contexte	Quel rôle l'IA doit jouer, sur quel sujet se concentrer ?	« Tu es PO d'un site de réservation hôtelière. »
Sortie attendue	Quel format, quelle structure, quelle longueur ?	« Une user story + 3 critères d'acceptation. »
Règles	Qu'est-ce qui est précis, mesurable, non négociable ?	« Cible : clients sur mobile. »
À éviter	Qu'est-ce qu'elle ne doit pas faire ou inclure ?	« N'invente pas de chiffres. »

Montrer un exemple du format attendu vaut mieux qu'une longue description : c'est le **few-shot**.

Modèle à copier :

```text
Tu es [rôle], tu travailles sur [sujet].
Produis [livrable], au format [structure, longueur].
Règles : [cible, contraintes, éléments obligatoires].
À éviter : [ce qu'il ne faut pas inventer ou inclure].
Exemple du format attendu : [un exemple court].
```

Clarifier avant de produire

1. **Besoin flou** : le besoin tel qu'on l'exprime à l'oral (« améliorer la réservation »).
2. **Brief structuré** : l'IA propose une structure (objectif, cible, contraintes, format), **vous la corrigez et la validez**.
3. **Livrable** : produit à partir du brief validé, pas du besoin flou.

La sortie d'une étape devient le **contexte** de la suivante. On ne demande pas tout d'un coup : on demande d'abord une structure vérifiable.

```text
On veut réduire les abandons de réservation sur mobile.
Propose-moi d'abord un brief : objectif, cible, contraintes,
format attendu. Ne rédige pas encore la user story.
```

Rewind plutôt qu'empiler

Quand une réponse part mal, la tentation est d'enchaîner les corrections (« non, pas comme ça… »). Chaque correction reste dans la conversation et le contexte devient de plus en plus confus.

Le bon réflexe : **revenir à la demande de départ** (bouton « Modifier » de votre message), la corriger, et relancer. Une mauvaise sortie est une **information sur l'entrée**, pas un texte à corriger.

Un bon brief devient un actif

Un brief qui marche sur **deux sujets différents** est réutilisable. Trois façons de le capitaliser :

1. **Instructions personnalisées** : toujours chargées (ton, langue, profil).
2. **Assistant personnalisé** : un espace dédié (GPT, Projet Claude, agent Copilot).
3. **Skill** : chargé **à la demande**, quand la tâche correspond (consignes, modèles, exemples).

Exemple d'assistant « User stories » :

```text
Tu es PO de l'équipe Réservation.
Pour chaque besoin que je te donne :
1. Propose d'abord un brief : objectif, cible, contraintes, format.
2. Attends ma validation.
3. Rédige la user story + 3 critères d'acceptation.
Ton clair, sans jargon. N'invente ni chiffres ni dépendances.
```

Attention au vocabulaire : les éditeurs appellent souvent ces assistants personnalisés des « agents ». Au sens strict, un agent est un modèle qui enchaîne lui-même des tools en boucle (voir la fiche 1).

À retenir

1. Écrivez votre demande avec les **4 ingrédients**, et montrez un exemple.
2. Demandez une **structure**, validez-la, puis demandez le livrable.
3. Une réponse ratée : **rewind**, pas une correction de plus.
4. Ce qui marche deux fois devient un **assistant** ou un **skill** d'équipe.

Mots-clés : prompt, brief structuré, few-shot, rewind, instructions personnalisées, assistant personnalisé, skill — voir le glossaire.

FICHE MODULE 3

Maîtriser le contexte

Ce que le modèle « voit » au moment de répondre, comment garder une session propre, et comment lui apporter le contexte de l'entreprise.

La context window

La **context window**, c'est **tout** ce que le modèle voit quand il répond :

- les **consignes** de l'assistant ;
- la **mémoire** (ce que l'assistant sait de vous) ;
- les **pièces jointes** ;
- la **conversation** : vos messages, ses réponses, vos corrections ;
- les **résultats des tools** (recherche web, fichiers lus, données d'un connecteur) ;
- l'**espace libre** restant. Quand il rétrécit, les réponses se dégradent et le coût monte.

La mémoire des assistants

Le modèle ne retient rien… mais l'**assistant** peut recopier des informations sur vous dans **chaque** conversation. Exemple : « Je suis PO, je travaille sur le parcours de réservation », dit une fois, est relu à chaque conversation.

- Elle influence les réponses **sans qu'on la voie** : une information périmée continue de peser.
- Deux collègues, même prompt, mémoires différentes : réponses différentes.
- Consultez-la, corrigez-la ou désactivez-la dans les **réglages** de votre assistant.

Plus n'est pas mieux

Session	Ce qu'elle contient	Effet
Simple	objectif, contraintes, un exemple	réponse ciblée
Chargée	historique long, documents, essais successifs	dérive douce
Polluée	infos périmées, erreurs, contradictions	réponses crédibles mais fausses
Préparée	repères choisis, demande claire	moins d'exploration inutile

La bonne question : **tout ce qui reste ici aide-t-il encore la tâche du jour ?**

Garder une session propre

1. **Avant : cadrer.** Une courte note de cadrage : objectif, documents utiles, règles, points de contrôle. L'IA peut la proposer ; vous arbitrez.
2. **Pendant : redémarrer.** Une tâche = une session. Si ça dérive, on ne corrige pas par-dessus : on repart avec la note de cadrage.
3. **Après : extraire.** Avant de fermer, on garde ce qui a de la valeur.

La règle : réponse ratée dès le début → **rewind** ; session longue qui dérive → **nouvelle session** + note de cadrage.

Capitaliser : de la session aux artefacts

Un **artefact** est un livrable qui vit **en dehors de la conversation** : on le relit, on le partage, on le réinjecte. Plutôt que « résume cette session », demandez une **extraction ciblée** :

Fichier	Contenu
`decisions.md`	arbitrages pris, sujets tranchés
`risques.md`	zones sensibles, dépendances
`actions.md`	prochaines étapes, responsables
`brief-suivant.md`	ce qu'on réinjecte la prochaine fois

```text
Extrais de cette conversation, en 3 listes courtes :
- les décisions prises,
- les risques identifiés,
- les prochaines actions (qui, quoi).
N'invente rien : si une liste est vide, dis-le.
```

Une extraction **non relue** n'est jamais une source d'autorité.

Apporter le contexte de l'entreprise

Le modèle ne connaît pas votre entreprise : ni vos CGV, ni vos process, ni vos données. Deux façons de le lui apporter :

RAG : vos documents. Le RAG (*Retrieval-Augmented Generation*) ancre les réponses dans vos sources ; on parle de **grounding**. Exemple : « Quelles sont les conditions d'annulation du tarif Flex ? »

1. **Préparer** : les documents internes sont indexés à l'avance, avec leurs droits d'accès.
2. **Chercher** : un tool trouve les passages pertinents (par le sens, pas seulement les mots).
3. **Ajouter** : ces passages, et eux seuls, entrent dans la context window.
4. **Répondre** : le modèle répond à partir des passages et cite ses sources.

Vigilance : des documents périmés donnent des réponses périmées… mais sourcées, donc convaincantes.

Tools / MCP : vos applications. Ils apportent des données fraîches et des actions (lire, créer, envoyer). Exemple : « Liste les réservations annulées hier et ouvre un ticket Jira. » Vigilance : l'agent agit avec **vos** accès.

Dans les deux cas, l'IA ne **sait** rien de plus : on lui **donne** le bon contexte, au bon moment.

À retenir

1. Le modèle ne voit que sa context window : choisissez ce qui y entre.
2. Une tâche = une session ; si ça dérive, repartez avec une note de cadrage.

3. Avant de fermer, extrayez décisions, risques et actions, et relisez-les.
4. Avec un RAG, demandez toujours les sources et vérifiez-les.

Mots-clés : context window, mémoire, session, note de cadrage, artefact, RAG, grounding, MCP — voir le glossaire.

FICHE MODULE 4

L'IA dans la discovery produit

Aller plus vite vers la preuve, sans sauter la preuve : où l'IA accélère le travail produit, et ce qui reste humain.

Mode projet ou mode produit ?

Mode projet	Mode produit
Un comité décide de tout refaire.	On mesure 38 % d'abandons au paiement mobile.
9 mois de développement : on livre le plan.	6 interviews et un prototype en 2 semaines : on teste l'hypothèse.
Au lancement, les abandons ne baissent pas.	Le test montre que le paiement en 1 clic marche.
On découvre trop tard que le problème était le paiement.	On construit ce qui a fait ses preuves.

L'IA rend le **mode produit** beaucoup moins cher : interviewer, synthétiser, prototyper prend des heures, et non plus des mois.

Interroger sur des faits, pas des opinions

Les clients prédisent mal leur propre comportement. Posez des questions sur ce qu'ils ont **fait**, pas sur ce qu'ils **pensent** qu'ils feraient.

À éviter	À la place
Opinion : « Quelles fonctionnalités voudriez-vous ? »	**Fait vécu** : « Racontez-moi la dernière fois que vous avez réservé. »
Hypothèse : « Utiliseriez-vous ceci si on le construisait ? »	**Chronologie** : « Reprenez depuis le début : que faisiez-vous juste avant ? »
Question orientée : « Ce n'est pas frustrant quand le paiement échoue ? »	**Creuser l'histoire** : « Pourquoi avez-vous fait comme ça ? »

Du terrain au PRD

Étape	Ce que c'est	L'IA aide à	Reste humain
Interviews	5 à 8 utilisateurs racontent des situations réelles	transcrire, regrouper les thèmes	parler à de vrais clients
Experience map	le parcours de bout en bout : étapes, ressentis, irritants	structurer la carte, repérer les irritants	choisir l'irritant à traiter

Étape	Ce que c'est	L'IA aide à	Reste humain
PRD	une page : quoi construire, pourquoi, comment mesurer	rédiger un premier jet	arbitrer objectif et périmètre
Prototype	une maquette cliquable pour tester avant de développer	générer une maquette en quelques heures	observer de vrais utilisateurs

Le PRD, c'est quoi ?

Le PRD (*Product Requirements Document*) est le contrat de l'équipe produit : **une page**, pas un cahier des charges. Il est écrit par le PO ou le PM avec le designer et le tech lead, après la discovery et avant de construire. Il dit **quoi** construire, **pourquoi**, et comment on mesure le succès ; le *comment* technique reste à l'équipe.

Modèle en 6 rubriques, avec l'exemple du parcours de réservation :

Rubrique	Exemple
Problème	38 % des réservations mobiles abandonnées au paiement.
Objectif	Passer de 38 % à 30 % d'abandons en 3 mois.
User story	En tant que client mobile, je veux payer en 1 clic pour finir ma réservation sans ressaisie.
Périmètre	Paiement mobile. Hors périmètre : desktop.
Hypothèse	La ressaisie de la carte bloque.
Critère de succès	Moins de 30 % d'abandons, 4 semaines de suite.

L'IA rédige un premier jet à partir des interviews ; l'équipe **arbitre** le problème, l'objectif et le périmètre. Relisez toujours ce qu'elle a inventé : un chiffre qui n'est pas dans vos sources doit être marqué « à mesurer ».

Prototyper : lo-fi d'abord

La **fidélité** (« fi ») mesure à quel point la maquette ressemble au produit final.

Niveau	À quoi ça ressemble	Temps	Sert à
Lo-fi	un croquis, des boîtes grises	quelques minutes	explorer, aligner l'équipe
Mid-fi	la structure et les vrais libellés, sans couleurs	quelques heures	valider le parcours
Hi-fi	presque le vrai produit, cliquable	quelques jours (quelques heures avec l'IA)	tester avec des utilisateurs

Trop beau trop tôt = changements chers trop tard. Avec l'IA, demandez explicitement une maquette « en niveaux de gris, sans couleurs » pour rester lo-fi au début.

Un prototype n'est pas un produit

« Il est génial ce prototype… on le met en prod la semaine prochaine ? » Non :

Prototype	Produit
Données fictives, sans droits d'accès	Données réelles : qui voit quoi, pas de fuite
Une démo, une personne	Des milliers d'utilisateurs, retour arrière possible
En cas d'erreur, on recharge la page	Accès contrôlés, erreurs détectées et suivies
On régénère	Une équipe le fait évoluer dans la durée

Avant toute démo, annoncez que c'est une **maquette**.

Tester, apprendre, recommencer

Le **user testing** consiste à observer de vrais utilisateurs, pas à leur demander ce qu'ils veulent :

- **Avant** : une question à laquelle répondre, 2 à 4 tâches réalistes, les bonnes personnes.
- **Pendant** : observer sans aider ; noter hésitations, clics ratés, abandons.
- **Après** : synthétiser à plusieurs, corriger, retester.

C'est une boucle : prototyper → tester → apprendre → nouvelle question. Exemple : le paiement en 1 clic marche… sauf avec une carte d'entreprise. C'est la **continuous discovery** : la discovery ne s'arrête pas au lancement.

À retenir

1. Interrogez sur des **faits vécus**, jamais sur des intentions.
2. Le PRD tient en **une page** : problème, objectif mesurable, périmètre, critère de succès.
3. Prototypez **lo-fi** d'abord, et testez avec de vrais utilisateurs.
4. Utilisez l'IA pour aller plus vite vers la preuve, pas pour sauter la preuve.

Mots-clés : product discovery, MVP, user interview, experience map, PRD, prototype lo-fi / hi-fi, user testing, continuous discovery — voir le glossaire.

FICHE MODULE 5

Risques, gouvernance et coûts

Ce qui peut mal tourner, les deux règles qui couvrent l'essentiel, et ce qui fait monter la facture.

5 risques, 5 garde-fous

Risque	En une phrase	Exemple	Garde-fou
Hallucination	une réponse fausse mais convaincante	des avocats sanctionnés pour avoir cité une jurisprudence inventée	relecture, grounding (sources vérifiables)
Prompt injection	un contenu piégé (document, email, page web) détourne l'agent	des données exfiltrées via un document partagé	accès de l'agent limités
Fuite de données	des informations sensibles sortent de l'entreprise	des secrets industriels collés dans un chatbot grand public	versions entreprise des assistants
Biais	des stéréotypes reproduits, parfois amplifiés	des biais régionaux mesurés dans les réponses de LLM testés	relire ce qui est sensible
Coûts	une consommation qui s'emballe	des clés d'accès volées, des factures à 5 chiffres	plafonds de dépense, suivi

2 règles d'or

1. Données sensibles : uniquement dans les versions entreprise. Rien de confidentiel dans un assistant grand public : clients, contrats, données personnelles, code. Les versions entreprise s'engagent par contrat à ne pas entraîner les modèles sur vos données. Les données personnelles restent soumises au RGPD : base légale, information des personnes, avis du DPO. *Le test : l'enverriez-vous par email à un inconnu ?*

2. Un humain valide avant d'agir. Obligatoire pour ce qui est irréversible, réglementé ou visible par un client. Cette validation est prévue dans le process, pas optionnelle, et faite par quelqu'un qui connaît le métier. *Le test : qui signe si c'est faux ?*

Ces deux règles traitent la **fuite** d'un côté et l'**erreur** de l'autre. La seconde s'appelle le **human-in-the-loop**. Plus vous branchez de documents et de tools sur un assistant, plus elles comptent : un agent agit avec vos droits.

Côté réglementation, l'AI Act européen impose depuis février 2025 que les personnes qui utilisent l'IA dans l'entreprise y soient formées (article 4), et le RGPD s'applique à toute donnée personnelle confiée à un assistant.

Coûts : le token se paie

- Les modèles se paient au **token**, en entrée comme en sortie.

- L'historique est **renvoyé à chaque message** : une longue conversation coûte de plus en plus cher.
- En abonnement, la limite prend la forme de quotas ; à l'usage (API, agents), elle prend la forme d'une facture. Dans les deux cas, une consommation élevée traduit souvent un **usage non optimisé**.

5 réflexes pour consommer moins :

1. **1 tâche = 1 session** : moins d'historique renvoyé.
2. **Être précis** : une demande vague déclenche des recherches inutiles.
3. **Couper les connecteurs (MCP) inutiles** : chacun pèse dans le contexte.
4. **Garder le même modèle** pendant une session : sinon le **cache** (le début du contexte déjà traité) est perdu.
5. **Demander des réponses concises** : moins de tokens en sortie.

À retenir

1. Rien de confidentiel hors des **versions entreprise**.
2. Un **humain qualifié valide** ce qui est irréversible, réglementé ou visible par un client.
3. Les sources se **vérifient** ; les accès des agents se **limitent**.
4. Sessions courtes et demandes précises : c'est meilleur pour la qualité **et** pour la facture.

Mots-clés : hallucination, prompt injection, fuite de données, biais, grounding, human-in-the-loop, token, cache — voir le glossaire.

SUPPORTS D'EXERCICES

Les exercices de la journée

Les consignes et les prompts de départ des exercices en salle. Faites-les dans l'assistant que vous utilisez habituellement (version entreprise), sur un sujet de votre métier si vous le pouvez.

Exercice 1 — Regenerate (4 min)

But : sentir que la réponse varie, et que quelques mots changent tout.

1. Écrivez une demande métier courte, volontairement **floue**.
2. Relancez-la **3 fois**. Notez ce qui reste stable.
3. Ajoutez **une seule** contrainte : cible, délai, canal ou format.
4. Relancez. Qu'est-ce qui a le plus bougé : le ton, la structure, ou carrément l'objectif ?

```text
Rédige une user story pour réduire les abandons de réservation.
```

puis :

```text
Rédige une user story pour réduire les abandons de réservation
sur mobile.
```

Autres points de départ : « Prépare une synthèse d'atelier pour une équipe produit » (+ « en 5 puces, pour un comité de direction ») ; « Rédige un email pour annoncer un changement de logiciel » (+ « pour des équipes terrain, en 120 mots max »).

Exercice 2 — Du flou au brief (15 min)

But : faire produire une structure, la corriger, puis seulement demander le livrable.

1. Partez d'un besoin métier **flou** de votre quotidien.
2. Demandez à l'IA un **brief structuré**, pas le livrable.
3. Corrigez **2 ou 3 champs**.
4. Demandez le **livrable** : user story, synthèse, plan d'action.

```text
On veut réduire les abandons de réservation sur mobile.
Propose-moi d'abord un brief : objectif, cible, contraintes,
format attendu. Ne rédige pas encore la user story.
```

puis, une fois le brief corrigé :

```text
Brief validé. Rédige maintenant la user story
avec 3 critères d'acceptation.
```

Pour le débrief : montrez le brief, pas le rendu. Qu'est-ce qu'il vous a forcé à clarifier ?

Exercice 3 — Extraire (10 min)

But : transformer une session en artefact réutilisable.

1. Prenez une conversation un peu **longue** : celle de l'exercice précédent, ou n'importe quelle conversation récente (préparation de réunion, rédaction d'un mail…).
2. Demandez une **extraction ciblée**, pas un résumé.
3. **Relisez** et corrigez : qu'est-ce qui manquait, qu'est-ce qui est faux ?
4. Gardez le résultat : c'est votre premier **artefact**.

```text
Extrais de cette conversation, en 3 listes courtes :
- les décisions prises,
- les risques identifiés,
- les prochaines actions (qui, quoi).
N'invente rien : si une liste est vide, dis-le.
```

Pour le débrief : l'extraction a-t-elle retrouvé l'essentiel ? Qu'avez-vous dû corriger ?

Exercice 4 — De l'interview au PRD (10 min)

Le support est dans le handout **« Exercice : de l'interview au PRD »**.

SUPPORT D'EXERCICE

Exercice : de l'interview au PRD

Interview fictive, menée dans le cadre de la discovery « abandons de réservation sur mobile ». Utilisez ce compte-rendu pour faire générer un PRD en 6 rubriques (problème, objectif, user story, périmètre, hypothèses, critères de succès), puis relisez-le.

Participante : Claire, 41 ans, cheffe de projet dans une entreprise de BTP. Voyage 2 à 3 fois par mois pour le travail, réserve presque toujours depuis son téléphone.

Contexte : entretien de 30 minutes en visio, mené par la PO et le designer de l'équipe Réservation.

« Racontez-moi la dernière fois que vous avez réservé un hôtel. »

> « Mardi dernier, dans le train. Je devais être à Lyon le lendemain pour un chantier. J'ai trouvé l'hôtel en deux minutes, ça c'est bien. Par contre au paiement… il a fallu ressaisir toute ma carte, alors que je réserve chez vous depuis des années. Dans le train, avec le réseau qui saute, la validation de la banque a expiré. J'ai dû tout recommencer. »

« Et ensuite, qu'avez-vous fait ? »

> « J'ai abandonné, honnêtement. J'ai réservé le soir depuis mon ordinateur à l'hôtel… enfin, chez moi. Mais le tarif avait augmenté de 15 euros. »

« Pourquoi pas tout de suite sur l'ordinateur ? »

> « Parce que je n'en ai pas dans le train. Et puis, je voulais régler ça tout de suite, sinon j'oublie. »

« Y a-t-il autre chose qui vous a gênée ? »

> « Au moment de payer, je ne savais plus si le tarif était annulable. Mes déplacements bougent souvent. C'était écrit quelque part plus haut, mais je n'avais pas envie de remonter. Ah, et je ne comprends jamais où mettre mon numéro de carte de fidélité. »

« Si vous pouviez changer une chose ? »

> « Payer avec la carte que vous avez déjà. Comme sur les sites de e-commerce. »

Notes de l'équipe

- Elle a utilisé le mot « recommencer » trois fois.
- Elle n'a pas parlé du prix, sauf pour les 15 euros perdus.
- Elle réserve pour son entreprise mais paie avec sa carte personnelle, puis se fait rembourser.

DÉMOS À REJOUER

Les démos

Les démos jouées par le formateur pendant la formation, pour les rejouer seul dans votre assistant, que vous ayez été présent ou non. Chaque démo dure quelques minutes et ne demande qu'un assistant de chat.

Le texte découpé en tokens

Module 1 · slide « Le texte découpé en tokens » · 1 min, optionnelle

Ce que ça montre : le modèle ne lit pas des mots mais des **tokens**. Les mots courants tiennent en un token, les mots rares sont découpés, et le français consomme plus de tokens que l'anglais.

À rejouer

1. Ouvrez un visualiseur de tokens en ligne (par exemple le *tokenizer* d'OpenAI).
2. Tapez un mot de votre métier, un nom de produit interne par exemple, et regardez en combien de morceaux il est découpé.
3. Tapez la même phrase en français puis en anglais, et comparez le nombre de tokens.

Ce qu'on observe : un mot rare ou un nom interne est découpé en plusieurs morceaux, et la phrase française compte plus de tokens que l'anglaise. Plus de tokens, c'est plus cher, plus lent, et plus de place prise dans la context window.

La souris mange…

Module 1 · démo « La souris mange… » · 3 min

Ce que ça montre : le modèle choisit la suite **la plus probable**, pas la plus vraie. Relancer change la forme, rarement l'idée ; un seul mot ajouté déplace toute la suite.

À rejouer

1. Avant de lancer, devinez vous-même la réponse.
2. Dans une conversation vide, collez :

```text
La souris mange …
Complète la phrase en une ligne.
```

3. Relancez 2 ou 3 fois (bouton « regénérer ») : notez ce qui change et ce qui reste.
4. Ajoutez **un seul mot** et relancez : « La souris **d'ordinateur** mange… », puis « La souris **de laboratoire** mange… ».
5. Comparez les réponses côte à côte.

Ce qu'on observe : la première réponse est celle que presque tout le monde devine, parce que c'est la suite la plus fréquente dans les textes, pas parce qu'elle est vraie (une souris mange surtout des graines). Les relances changent la formulation, pas l'idée. Le mot ajouté, lui, change toute la réponse : chaque mot de votre demande pèse.

Clarifier avant de produire

Module 2 · slide « Clarifier avant de produire » · 2 min, optionnelle

Ce que ça montre : on demande d'abord une **structure** (le brief), on la corrige, *puis* le livrable. Le brief validé devient le contexte de l'étape suivante.

À rejouer

1. Demandez un brief plutôt que le livrable :

```text
On veut réduire les abandons de réservation sur mobile.
Propose-moi d'abord un brief : objectif, cible, contraintes,
format attendu, points de vigilance. Ne rédige pas encore
la user story.
```

2. Comparez avec ce brief de référence, puis corrigez le vôtre :

```text
objectif: réduire les abandons sur mobile
public_cible: clients qui réservent depuis leur téléphone
contrainte: amélioration visible avant la prochaine release
format_attendu: user story avec critères d'acceptation
points_de_vigilance:
  - ne pas inventer de dépendance technique
  - rester compréhensible par l'équipe produit
```

3. Une fois le brief corrigé, demandez le livrable : « Brief validé. Rédige maintenant la user story avec 3 critères d'acceptation. »

Ce qu'on observe : le brief fait apparaître ce que l'IA aurait inventé (une cible, un délai, une contrainte technique). Le livrable produit à partir du brief corrigé est plus juste que celui produit directement depuis le besoin flou.

FICHE MÉMO

Cheat sheet : 7 réflexes pour lundi matin

Une page à garder sous la main.

Les 7 réflexes

1. **Relancer** avant de faire confiance : une réponse est une proposition.
2. Écrire sa demande avec les **4 ingrédients** (rôle, sortie attendue, règles, à éviter)… et transformer ce qui marche en **skill** ou assistant d'équipe.
3. Faire produire une **structure**, la valider, puis demander le livrable, **avec un exemple** du format attendu.
4. **Repartir proprement** quand la session dérive : rewind si ça part mal dès le début, nouvelle session et note de cadrage si une longue session dérive.
5. **Extraire** décisions, risques et actions avant de fermer, et relire l'extraction.
6. En discovery : **interviewer sur des faits**, prototyper **lo-fi**, **tester** avec de vrais utilisateurs.
7. **Pas de données sensibles** hors des versions entreprise des assistants, et un **humain qui valide** avant d'agir.

La boucle

Tester (relancer, comparer) → **Structurer** (4 ingrédients, brief validé) → **Cadrer le contexte** (note de cadrage, bons documents, session propre) → **Capitaliser** (artefacts) → on repart sur une base propre.

À chaque étape : **choisir ce qu'on donne** au modèle, **relire avec un œil humain**.

Les questions à se poser

- La sortie varie beaucoup ? → *Qu'est-ce que je n'ai pas cadré ?*
- La session s'allonge ? → *Tout ce qui reste ici aide-t-il encore la tâche du jour ?*
- L'IA cite un chiffre ? → *D'où vient-il ? Est-il dans mes sources ?*
- Je colle un document ? → *L'enverrais-je par email à un inconnu ?*
- L'IA va agir à ma place ? → *Qui signe si c'est faux ?*

Le modèle de demande

text

```
Tu es [rôle], tu travailles sur [sujet].
Produis [livrable], au format [structure, longueur].
Règles : [cible, contraintes, éléments obligatoires].
À éviter : [ce qu'il ne faut pas inventer ou inclure].
Exemple du format attendu : [un exemple court].
```

GLOSSAIRE

Glossaire des mots-clés IA

Une phrase par terme, sans jargon. Les termes techniques anglais sont gardés : ce sont ceux que vous entendrez dans les outils et les réunions.

Les bases (module 1)

Terme	En une phrase
IA générative	IA qui produit du contenu (texte, image, code) au lieu de seulement classer ou prédire un chiffre.
LLM (Large Language Model)	Modèle entraîné sur d'énormes volumes de texte, qui génère une réponse en prédisant le mot suivant.
Token	Morceau de mot traité par le modèle ; c'est aussi l'unité de facturation (≈ ¾ de mot en anglais).
Hallucination	Réponse fausse mais présentée avec assurance et vraisemblance.
Knowledge cutoff (date de coupure)	Date à laquelle s'arrêtent les données d'entraînement du modèle ; il ignore tout ce qui s'est passé après, sauf s'il fait une recherche web.
Multimodal (LMM)	Modèle qui comprend aussi images, audio, documents, pas seulement du texte.
Assistant	Le produit qu'on utilise au quotidien (ChatGPT, Copilot, Claude.ai, Gemini) : une interface posée sur un harness, autour d'un modèle.
Harness	Tout le logiciel autour du modèle : consignes, mémoire, tools, enchaînement des étapes. Deux assistants sur le même modèle diffèrent par leur harness.
Tools	Capacités que le modèle peut appeler pendant sa réponse : recherche web, calcul, lecture de fichiers, Jira, Figma… (*tool calling*).
MCP (Model Context Protocol)	Standard pour brancher de nouveaux tools sur un assistant : applications et données de l'entreprise.
Agent	Modèle qui enchaîne lui-même des actions avec des tools, en boucle, jusqu'à atteindre un objectif : c'est la **boucle agentique** (modèle + tools + boucle + objectif, que le harness fait tourner). Les éditeurs utilisent aussi « agent » pour de simples assistants personnalisés.
Boucle agentique (agent loop)	Le cycle demande → le modèle choisit un tool → le harness l'exécute → le résultat entre dans le contexte → on recommence si besoin.

Bien demander (module 2)

Terme	En une phrase
Prompt	La demande envoyée au modèle, consignes comprises.
Brief structuré	Demande découpée en champs explicites : objectif, cible, contraintes, format attendu, critères.
Few-shot	Donner un ou plusieurs exemples du résultat attendu dans la demande.
Format de sortie	La forme imposée à la réponse : tableau, liste en 5 points, user story, JSON…
Rewind	Revenir corriger la demande de départ plutôt qu'empiler des corrections dans la conversation.
Instructions personnalisées / assistant personnalisé	Consignes enregistrées une fois et réappliquées : toujours (instructions personnalisées) ou dans un espace dédié (GPT, Projet Claude, agent Copilot).
Skill	Savoir-faire emballé (consignes, modèles, exemples, parfois des scripts) que l'assistant charge **à la demande**, quand la tâche correspond. Plus léger que des instructions permanentes, partageable dans une équipe. Claude parle d'« Agent Skills » ; d'autres assistants ont des équivalents.
Vibe coding	Produire vite un prototype en langage naturel, sans regarder le code ; parfait pour explorer, insuffisant pour livrer.
AI-driven development	Développement logiciel où l'IA écrit beaucoup de code, sous la direction et le contrôle d'ingénieurs.

Le contexte (module 3)

Terme	En une phrase
Context window	Tout ce que le modèle « voit » au moment de répondre : consignes, historique, pièces jointes, résultats des tools.
Session	Une conversation ; elle accumule du contexte, utile ou parasite.
Mémoire	Informations que l'assistant réinjecte automatiquement d'une session à l'autre (profil, préférences) ; souvent désactivable.
Dérive de contexte (côté dev : context rot)	Session qui se dégrade quand la context window se remplit d'ajouts, de corrections et d'informations périmées ou contradictoires : elle reste crédible mais s'éloigne du besoin initial.

Terme	En une phrase
Artefact	Livrable produit avec l'IA qui vit en dehors de la conversation (note, synthèse, `decisions.md` …) : on le relit, on le partage, on le réinjecte. Les assistants en ont souvent une vue dédiée (Artifacts dans Claude, Canvas dans ChatGPT ou Copilot).
Note de cadrage	Court document préparé avant une tâche : objectif, sources utiles, règles, points de contrôle.
Grounding	Ancrer la réponse dans des sources vérifiables plutôt que dans la « mémoire » du modèle.
RAG (Retrieval-Augmented Generation)	Technique de grounding : on cherche les bons extraits de documents internes et on les ajoute au contexte avant de répondre.

Discovery produit (module 4)

Terme	En une phrase
Product discovery	Phase où l'on cherche *quoi* construire, en testant les hypothèses avant d'investir.
Continuous discovery	Faire de la discovery en continu, pas seulement en début de projet.
User interview	Entretien sur un comportement passé réel, pas sur une opinion ou une intention.
Experience map	Visualisation du parcours complet d'une personne pour atteindre un objectif, avec ses irritants.
JTBD (Job To Be Done)	Le « travail » qu'un utilisateur cherche à accomplir, indépendamment de la solution.
MVP	Plus petite version qui permet de tester l'hypothèse la plus risquée.
PRD (Product Requirements Document)	Une page, écrite par le PO/PM avec designer et tech lead, qui dit quoi construire, pourquoi, et comment on mesure le succès. Ce n'est pas une spec technique : le *comment* reste à l'équipe.
Prototype lo-fi / mid-fi / hi-fi	« fi » = *fidelity*, la ressemblance avec le produit final. Lo-fi : un croquis en boîtes grises ; mid-fi : la structure et les vrais libellés, sans couleurs ; hi-fi : presque le vrai produit, cliquable. On commence lo-fi.
User testing	Observer de vrais utilisateurs manipuler un prototype pour voir où ils bloquent.

Risques et coûts (module 5)

Terme	En une phrase
Human-in-the-loop	Un humain qualifié valide la sortie de l'IA *avant* qu'elle soit utilisée.
Prompt injection	Instructions malveillantes cachées dans un contenu (email, document, page web) qui détournent un assistant.
Fuite de données	Données confidentielles envoyées à un assistant non maîtrisé (version grand public).
Biais	Tendance du modèle à reproduire ou amplifier des déséquilibres présents dans ses données.
Cache	Mise en mémoire du début du contexte pour ne pas le repayer plein tarif à chaque message (il est relu à prix réduit) ; changer de modèle ou de tools en cours de route le perd.
Plafond de dépense (spend cap)	Limite de budget configurée pour éviter les factures surprises.